

Restaurant Intelligent

Explication complète du code

Node.js · Express · Socket.IO + Vision IA Python (OpenCV / YOLOv8)

Ce document explique, fichier par fichier, comment fonctionne l'application **Restaurant Intelligent** : un serveur Node.js temps réel relié à un module de vision par ordinateur Python qui détecte l'occupation des tables, plus trois interfaces web (client par QR Code, cuisine/serveur, administration).

Document généré automatiquement — projet C : \wammpp64\www\restaurant

1. Vue d'ensemble

L'application met en relation **trois mondes** :

- **Le client** scanne un QR Code posé sur sa table, ouvre le menu sur son téléphone et passe commande, **sans créer de compte**.
- **Le personnel** (cuisine / serveur) reçoit les commandes **en temps réel** et visualise le plan de salle (tables libres / occupées).
- **L'IA Python** analyse le flux d'une caméra, détecte les personnes assises et envoie au serveur l'état de chaque table.

Le tout repose sur **un seul serveur Node.js** qui sert à la fois l'API REST, les pages web statiques et le canal temps réel (WebSocket via Socket.IO).

Schéma d'architecture

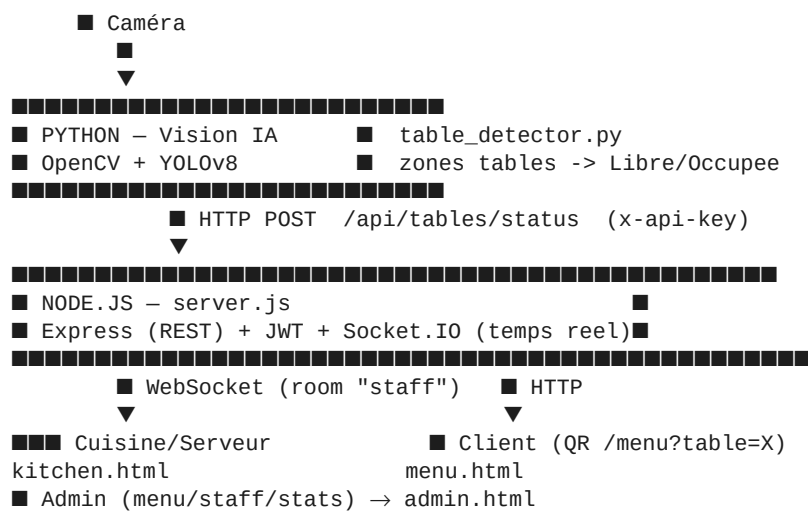


Figure 1 — La caméra alimente l'IA, l'IA et les clients alimentent le serveur Node, qui pousse les mises à jour vers le staff en temps réel.

2. Structure des fichiers

Fichier	Rôle
server.js	Serveur principal : API REST, auth JWT, Socket.IO, génération QR
data/db.js	« Base de données » en mémoire (users, menu, orders, tables)
public/index.html	Page d'accueil (portail vers les 4 espaces)
public/menu.html	Espace CLIENT — menu + panier (accès par QR Code)
public/kitchen.html	Espace STAFF — commandes temps réel + plan de salle
public/admin.html	Espace ADMIN — menu, personnel, statistiques
public/qrcodes.html	Génération / impression des QR Codes des tables
ai_vision/table_detector.py	Module IA : détection des personnes par table
package.json	Dépendances Node et scripts (start / dev)
.env.example	Variables d'environnement (PORT, secrets)

3. Le serveur Node.js — server.js

C'est le cœur du projet. Il utilise **Express** pour l'API, **Socket.IO** pour le temps réel, **jsonwebtoken** pour l'authentification et **qrcode** pour générer les QR Codes.

3.1 Initialisation

```
const app = express();
const server = http.createServer(app);
const io = new Server(server, { cors: { origin: "*" } });

app.use(cors());
app.use(express.json({ limit: "6mb" })); // 6mb : photos en Data URL
app.use(express.static(path.join(__dirname, "public")));
```

On crée un serveur HTTP commun à Express et à Socket.IO. La limite JSON est montée à 6 Mo pour accepter les images de plats envoyées en *Data URL* depuis l'admin. Le dossier `public/` est servi en statique.

3.2 Authentification & rôles (3 middlewares)

```
function authRequired(req, res, next) { // 1) Token JWT valide ?
  const token = (req.headers.authorization||"").replace("Bearer ", "");
  try { req.user = jwt.verify(token, JWT_SECRET); next(); }
  catch { return res.status(401).json({ error: "Token invalide" }); }
}

function authorize(...roles) { // 2) Bon rôle ?
  return (req, res, next) =>
    roles.includes(req.user.role) ? next()
    : res.status(403).json({ error: "Accès interdit" });
}

function aiKeyRequired(req, res, next) { // 3) Clé API de l'IA ?
  if (req.headers["x-api-key"] !== AI_API_KEY)
    return res.status(401).json({ error: "Clé API IA invalide" });
  next();
}
```

- **authRequired** : vérifie le jeton JWT envoyé dans l'en-tête `Authorization: Bearer ...` et place l'utilisateur décodé dans `req.user`.
- **authorize(...roles)** : autorise seulement certains rôles (ex. `authorize("admin")`).
- **aiKeyRequired** : protège l'endpoint de l'IA par une clé secrète partagée (`x-api-key`), différente du JWT car le script Python n'est pas un utilisateur connecté.

3.3 Connexion — /api/auth/login

```
const user = users.find(u => u.username === username);
if (!user || !bcrypt.compareSync(password, user.passwordHash))
  return res.status(401).json({ error: "Identifiants incorrects" });

const token = jwt.sign({ id, username, role }, JWT_SECRET, { expiresIn: "8h" });
res.json({ token, role, username });
```

Le mot de passe est comparé à son **hash bcrypt** (jamais stocké en clair). Si c'est bon, on signe un JWT valable 8 h contenant l'id, le nom et le rôle.

3.4 Commande client — POST /api/orders

Étape sensible : on ne fait **jamais confiance aux prix envoyés par le client**. La commande est reconstruite à partir du menu officiel côté serveur (anti-triche).

```
for (const it of items) {
  const dish = menu.find(m => m.id === it.id && m.available);
  if (!dish) continue; // plat inconnu/indispo -> ignoré
  const qty = Math.max(1, parseInt(it.qty) || 1);
  lineItems.push({ id: dish.id, name: dish.name, price: dish.price, qty });
}
const total = lineItems.reduce((s, i) => s + i.price * i.qty, 0);
orders.push(order);
io.to("staff").emit("order:new", order); // ■ temps réel -> cuisine
```

Une fois la commande créée, `io.to("staff").emit(...)` la pousse **instantanément** à tout le personnel connecté, sans rechargement de page.

3.5 État des tables — POST /api/tables/status (appelé par l'IA)

```
app.post("/api/tables/status", aiKeyRequired, (req, res) => {
  for (const t of req.body.tables) {
    tables[t.table].status = t.status === "Occupée" ? "Occupée" : "Libre";
    tables[t.table].updatedAt = new Date().toISOString();
  }
  io.to("staff").emit("tables:update", updated); // ■ plan de salle live
});
```

Protégé par `aiKeyRequired`, cet endpoint reçoit la liste des tables et leur état depuis Python, puis diffuse la mise à jour au staff.

3.6 QR Codes — /api/qrcode

```
function getLanIp() { /* cherche l'IPv4 locale non interne */ }

app.get("/api/qrcode", async (req, res) => {
  const url = `${base}/menu?table=${table}`; // ce qu'encode le QR
  const png = await QRCode.toBuffer(url, { width: 300, margin: 2 });
  res.type("png").send(png);
});
```

Le serveur détecte son IP sur le réseau local (LAN) pour que les téléphones des clients puissent l'atteindre, puis génère un PNG de QR Code encodant `/menu?table=X`.

3.7 Statistiques — GET /api/stats (admin)

```
const revenue = orders.reduce((s, o) => s + o.total, 0);
const occupied = Object.values(tables).filter(t => t.status==="Occupée").length;
// top 5 des plats les plus vendus, agrégés depuis toutes les commandes
res.json({ totalOrders, revenue, tablesOccupied, tablesTotal, topDishes });
```

Calcule le chiffre d'affaires, le nombre de tables occupées et le classement des plats les plus commandés.

4. Le temps réel — Socket.IO

Socket.IO maintient une connexion **WebSocket** ouverte entre le serveur et chaque navigateur du personnel. Le serveur peut ainsi « pousser » des événements sans que le client ait à demander.

4.1 Authentification du socket

```
io.use((socket, next) => { // garde d'entrée
  const token = socket.handshake.auth?.token; // JWT passé à la connexion
  try { socket.user = jwt.verify(token, JWT_SECRET); next(); }
  catch { next(new Error("Auth socket refusée")); }
});

io.on("connection", (socket) => {
  if (["admin", "serveur"].includes(socket.user.role)) {
    socket.join("staff"); // rejoint la "room" du staff
    socket.emit("tables:snapshot", tables); // état initial
    socket.emit("orders:snapshot", orders);
  }
});
```

Seuls les sockets présentant un JWT valide sont acceptés. Le staff rejoint la **room** « **staff** » ; à la connexion, on lui envoie un *snapshot* de l'état courant pour qu'il démarre avec les bonnes données.

4.2 Les événements échangés

Événement	Sens	Contenu
order:new	serveur → staff	Nouvelle commande client
order:update	serveur → staff	Changement de statut d'une commande
tables:update	serveur → staff	Nouvel état des tables (depuis l'IA)
tables:snapshot	serveur → staff	État complet des tables à la connexion
orders:snapshot	serveur → staff	Toutes les commandes à la connexion

5. Les données — data/db.js

Pour la démo, les données vivent **en mémoire** (de simples tableaux JS). Elles sont réinitialisées à chaque redémarrage du serveur. En production, on remplacerait ce fichier par PostgreSQL ou MongoDB.

```
const users = [ // mots de passe hashés au démarrage avec bcrypt
  { id:1, username:"admin", role:"admin", passwordHash: hash("admin123") },
  { id:2, username:"serveur", role:"serveur", passwordHash: hash("serveur123") },
];
const menu = [ { id:1, name:"Salade César", category:"Entrée", price:6.5, ... } ];
const orders = []; // rempli par les clients
const tables = { 1:{status:"Libre"}, 2:{...}, 3:{...} }; // mis à jour par l'IA

Identifiants de démo : admin / admin123 — serveur / serveur123
```

6. La vision par ordinateur — table_detector.py

Ce script Python indépendant lit une caméra (ou une vidéo), détecte les personnes avec **YOLOv8** et déduit quelles tables sont occupées.

6.1 Configuration

```
NODE_URL = "http://localhost:3000/api/tables/status"
API_KEY = "ai-secret-key" # doit = AI_API_KEY côté Node
CONF_THRESHOLD = 0.45 # confiance minimale YOLO
PERSON_CLASS_ID = 0 # 'person' dans le dataset COCO
TABLE_ZONES = { # rectangles (x1,y1,x2,y2) en pixels
    1: (40, 60, 220, 300),
    2: (240, 60, 420, 300),
    3: (440, 60, 620, 300),
}
MODEL_PATH = "yolov8n.pt" # modèle nano, téléchargé au 1er run
```

Chaque table est définie par un **rectangle de pixels** dans l'image. Le modèle `yolov8n.pt` est pré-entraîné sur COCO et contient déjà la classe *person* : **aucun dataset à télécharger**.

6.2 Boucle principale

```
results = model(frame, classes=[PERSON_CLASS_ID], conf=CONF_THRESHOLD)
table_states = {t: "Libre" for t in TABLE_ZONES} # tout libre par défaut

for box in results[0].boxes: # chaque personne détectée
    x1, y1, x2, y2 = box.xyxy[0].tolist()
    foot_x = int((x1 + x2) / 2) # centre horizontal
    foot_y = int(y2) # bas de la box = pieds
    for table, zone in TABLE_ZONES.items():
        if point_in_zone(foot_x, foot_y, zone):
            table_states[table] = "Occupée" # personne dans la zone
```

Astuce clé : on teste le **point au sol** (bas-centre de la boîte englobante = les pieds), pas le centre du corps. C'est plus fiable pour savoir à quelle table appartient réellement la personne.

6.3 Envoi temps réel (anti-spam)

```
now = time.time()
if table_states != last_states or (now - last_sent) > SEND_INTERVAL:
    send_status(table_states) # POST vers Node avec x-api-key
    last_sent = now
    last_states = dict(table_states)
```

On n'envoie au serveur **que si l'état a changé**, ou au minimum toutes les 1,5 s. Cela évite d'inonder le serveur à chaque image (≈ 30 par seconde).

7. Rôles et tableau des routes API

Les trois rôles

Rôle	Authentification	Accès
Admin	JWT — admin / admin123	Menu, personnel, statistiques
Serveur / Cuisine	JWT — serveur / serveur123	Commandes temps réel + plan de salle
Client	Aucune (QR Code)	Voir le menu, passer commande

Routes principales

Méthode	Route	Accès	Description
POST	/api/auth/login	public	Connexion (renvoie un JWT)
GET	/api/menu	public	Liste des plats disponibles
POST	/api/orders	client	Valider une commande
GET	/api/orders	staff	Liste des commandes
PATCH	/api/orders/:id/status	staff	Changer le statut
POST	/api/tables/status	IA (clé)	Mise à jour des tables
GET	/api/tables	staff	État des tables
GET	/api/stats	admin	Statistiques
GET/POST/PATCH/DELETE	/api/admin/menu	admin	Gérer le menu
GET/POST/DELETE	/api/admin/staff	admin	Gérer le personnel
GET	/api/qrcode?table=X	public	Image PNG du QR Code

8. Démarrage

Backend Node.js

```
npm install
copy .env.example .env      # Windows — puis ajuster les secrets
npm start                   # -> http://localhost:3000
```

Module IA Python

```
cd ai_vision
pip install -r requirements.txt
python table_detector.py          # webcam
python table_detector.py --source demo.mp4
# 'q' pour quitter la fenêtre OpenCV
```

Points de sécurité à renforcer en production

- Remplacer la base en mémoire par une vraie base (PostgreSQL / MongoDB).
- Mettre JWT_SECRET et AI_API_KEY dans .env (jamais en dur).
- Passer en HTTPS et restreindre le CORS.
- Limiter le débit (rate-limit) sur /api/orders et /api/tables/status.

Fin du document — Restaurant Intelligent